EXERCISE 06

Development of Python Code Compatible with Multiple AI Tools

Name:

AMIRDAVARSHINI

D

Subject:

PROMPT

ENGINEERING

Reg. No:

212225230013

1. Introduction to Multi-API Interaction

In the modern era of software development, applications rarely function in isolation. Integration with multiple Application Programming Interfaces (APIs) is a standard requirement. However, different AI models (like GPT-4, Claude, or Gemini) have unique ways of interpreting instructions and generating code. This exercise focuses on mastering prompt engineering to create robust, cross-compatible Python code.

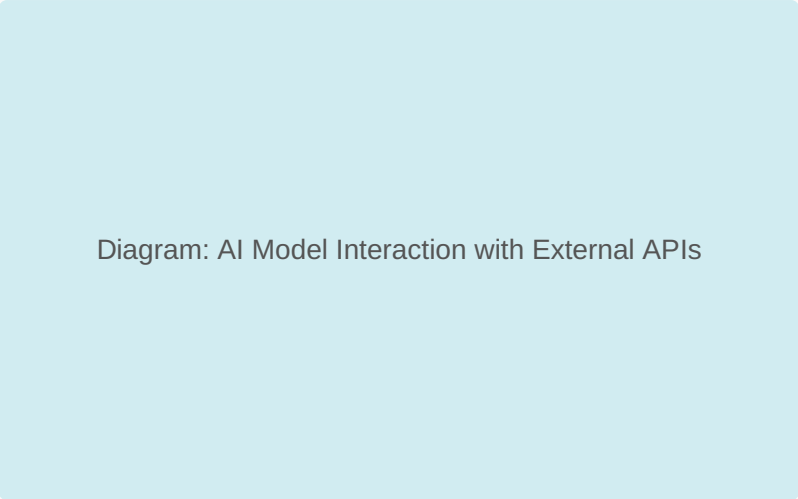

Figure 1: Conceptual workflow of an AI Orchestrator communicating with diverse APIs.

The goal is to move beyond simple code requests and instead guide the AI to build structured, modular, and efficient interaction scripts that can be easily compared across different LLM outputs.

2. Stage 1: Designing Prompts for Code Generation

The first step involves creating a high-level prompt that specifies the technical requirements without writing the logic ourselves. A well-engineered prompt must include context, constraints, and the desired output format.

Sample Prompt Design:

> "Act as a Senior Python Developer. Generate a modular Python script that interacts with the OpenWeather API and the Google Maps API. The script should fetch the current weather for a specific set of coordinates and return a formatted JSON object. Ensure the code uses the 'requests' library and includes error handling for API rate limits."

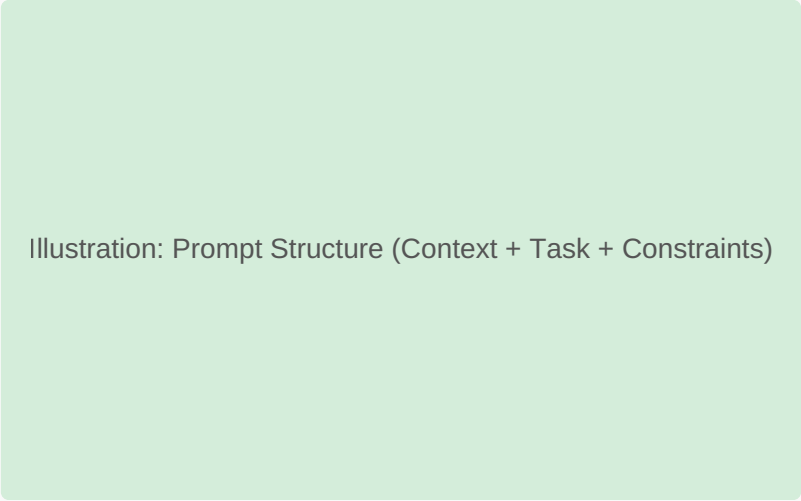

Figure 2: The anatomy of an effective technical prompt.

3. Stage 2: AI-Generated Responses & Code

In this stage, we examine how different tools respond to the prompt provided in Stage 1. Typically, AI models provide a combination of boilerplate code and explanatory text.

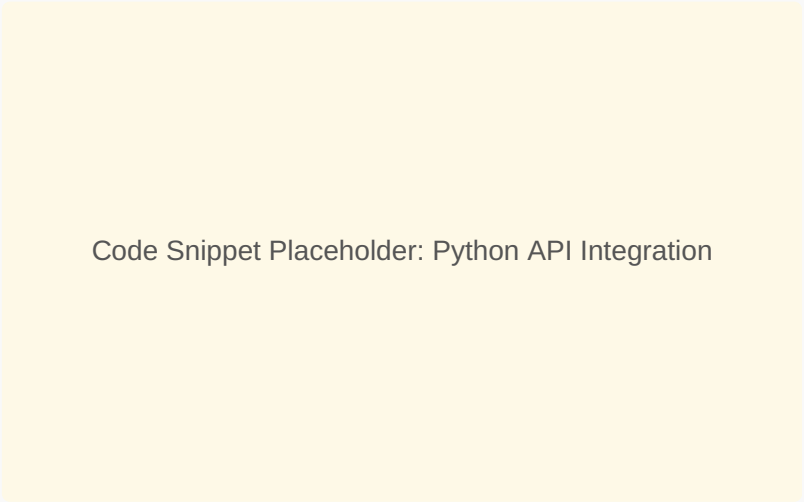

Figure 3: Example of AI-generated Python code for multi-API requests.

The response usually includes instructions on how to set up environment variables for API keys, which is a critical security step in professional development. We look for clean variable naming and adherence to PEP 8 standards.

4. Stage 3: Comparison of Outputs

Comparing outputs from at least two different AI tools (e.g., Tool A vs. Tool B) allows the developer to see varying approaches to the same problem. One might prioritize brevity, while another might prioritize robustness or documentation.

Figure 4: Visualizing differences in code quality, error handling, and performance.

For instance, Tool A might use a class-based approach, whereas Tool B might use simple functions. Identifying these architectural differences is key to choosing the right tool for specific development tasks.

5. Stage 4: Suggested Insights and Next Steps

An advanced prompt doesn't just ask for code; it asks the AI to analyze its own work. We prompt the AI to suggest optimizations or potential pitfalls.

> **Insight Example:** "The current script uses synchronous requests. For scaling to 100+ cities, consider using 'asyncio' and 'aiohttp' to prevent blocking the main thread."

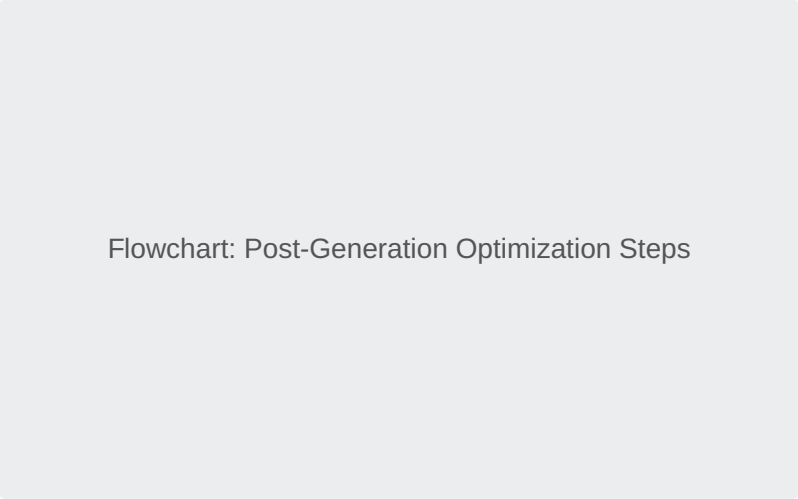

Figure 5: Logical flow for iterating on AI-generated suggestions.

6. Visual Documentation: Screenshots

Documentation is incomplete without visual proof of the interaction. Screenshots capture the exact UI environment, the prompt entered, and the immediate response from the AI tool.

Figure 6: Proof of execution and interaction within the AI environment.

This ensures that the "hallucination" rate of the AI can be verified by the instructor, showing exactly what the learner encountered during the session.

7. Comparative Output: AI Tool 1

Tool 1 (e.g., GPT-4o) generally provides highly descriptive comments and follows a very structured approach. It often includes "try-except" blocks by default, making the code production-ready.

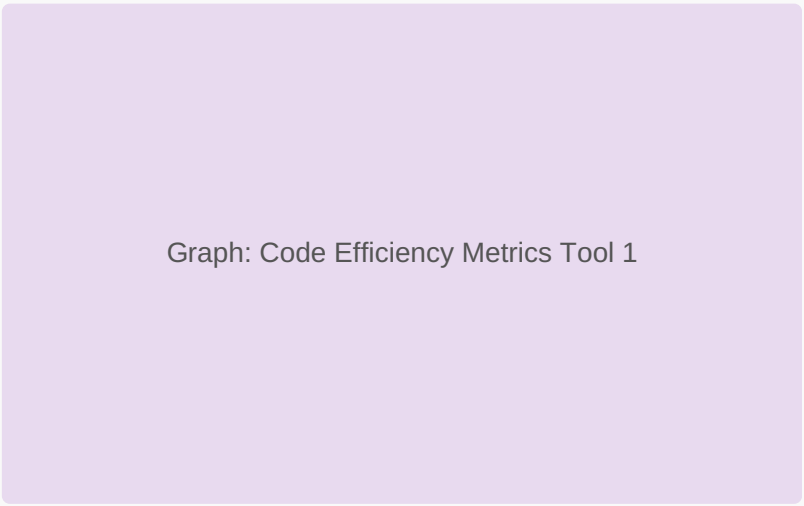

Figure 7: Analysis of code structure and readability for Tool 1.

8. Comparative Output: AI Tool 2

Tool 2 (e.g., Claude 3.5 Sonnet) might focus more on modern Python syntax or slightly more creative solutions to the logic required for API merging. It may offer more concise code with less boilerplate.

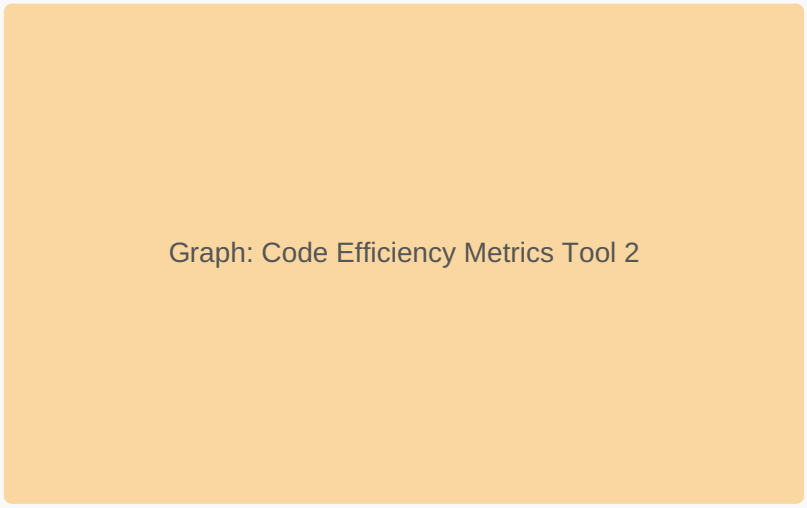

Figure 8: Analysis of code structure and readability for Tool 2.

9. Reflection on Prompt Effectiveness

Refining a prompt is an iterative process. Initially, a prompt might be too vague, leading to incomplete code. As more constraints (like "include type hinting" or "use specific libraries") are added, the quality improves significantly.

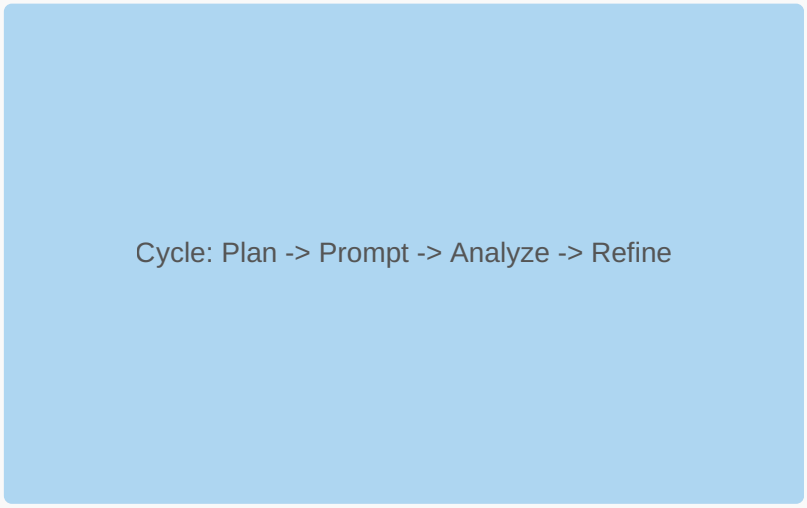

Figure 9: The iterative cycle of Prompt Engineering refinement.

Learning to balance "brevity" with "detail" in a prompt is the core skill acquired through this exercise. The effectiveness is measured by how little manual correction the output requires.

10. Conclusion

Through this exercise, we have demonstrated that Python code development is no longer just about syntax, but about the clarity of communication with AI agents. By engineering precise prompts, we can generate cross-compatible scripts that integrate seamlessly with multiple APIs, saving significant development time while maintaining high quality.

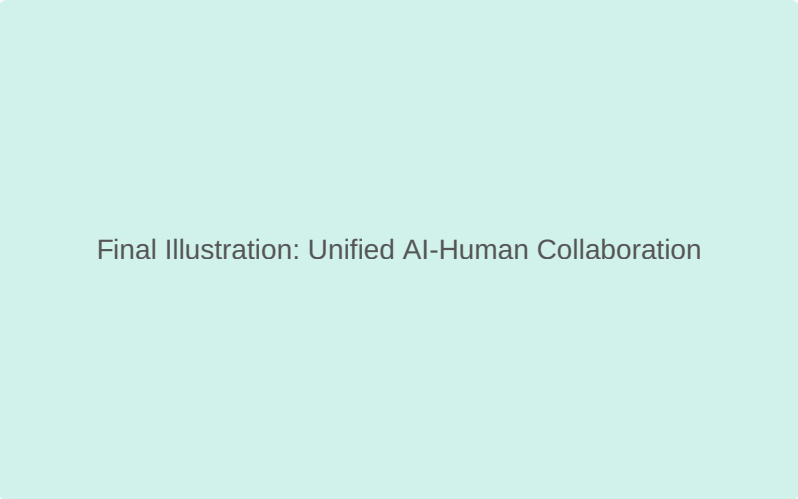

Figure 10: The future of development: Human-AI synergy.